

```
#include <stdio.h>
```

```
#include <mpi.h>
```

```
int main(int argc, char *argv[])
```

```
{
```

```
    int rank, size;
```

```
    int send_buf = 100, recv_buf;
```

```
    MPI_Init(&argc, &argv);
```

```
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
```

```
    MPI_Comm_size(MPI_COMM_WORLD, &size);
```

```
    if (size < 2) {
```

```
        if (rank == 0) {
```

```
            printf("This program requires at least 2 processes.\n");
```

```
        }
```

```
        MPI_Finalize();
```

```
        return 0;
```

```
    }
```

```
/*
```

```
 * UNCOMMENT THIS BLOCK TO DEMONSTRATE DEADLOCK:
```

```
 * Both processes wait for each other to receive before sending!
```

```
 *
```

```
 * if (rank == 0) {
```

```
 *     MPI_Recv(&recv_buf, 1, MPI_INT, 1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```

```
 *     MPI_Send(&send_buf, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
```

```
 * } else if (rank == 1) {
```

```
 *     MPI_Recv(&recv_buf, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```

```

*   MPI_Send(&send_buf, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);
* }
*/

// DEADLOCK AVOIDANCE: Altering the call sequence
if (rank == 0) {
    // Process 0 Sends FIRST, then Receives
    MPI_Send(&send_buf, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
    printf("Process 0 sent data to Process 1\n");

    MPI_Recv(&recv_buf, 1, MPI_INT, 1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    printf("Process 0 received data from Process 1\n");
}
else if (rank == 1) {
    // Process 1 Receives FIRST, then Sends
    MPI_Recv(&recv_buf, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    printf("Process 1 received data from Process 0\n");

    MPI_Send(&send_buf, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);
    printf("Process 1 sent data to Process 0\n");
}

MPI_Finalize();
return 0;
}

```